

External legibility made executable

Polylogue + Sinex

A proof-first public story, an independent oracle corpus, repository patches, a dependency-aware mission graph, and a safer single-machine swarm control plane.

Static repository access - 10 July 2026

01 A command that proves
the thesis

02 An oracle independent
of the products

03 A mission DAG agents
can execute

04 A transcript-complete
backend contract

The first edition described the right destination. This edition adds the control system.

The gap was not more prose. It was executable proof, independent ground truth, and coordination that survives parallel agent work.

From strategy to proof

A new `polylogue demo receipts` command traverses the real parser, storage, query, and ref path. It compares assistant prose with typed tool evidence, later repair, and a negative control.

From demo ideas to experiments

Incident 14:32 freezes source materials and an oracle outside either product. Every flagship demo now declares its construct, oracle, controls, falsifier, non-claims, and regeneration path.

From swarm prose to a scheduler

Twenty-three missions form an acyclic 72h / 7d / 30d graph. Leases, heartbeats, resource capacities, write-path exclusion, and mandatory handoffs make it runnable on one machine.

The public story can now be audited against commands, files, hashes, Beads owners, and explicit stop conditions.

Two products, one intelligible stack.

The backend relationship becomes maximal without collapsing the domain boundaries.

Polylogue

The local flight recorder and system of record for AI work.

Owns provider normalization, transcript semantics, lineage, AI-work query, reviewed memory, context compilation, and product UX.

Sinex

The local, replayable evidence substrate for digital life and agent work.

Owns durable material, evidentiary lifecycle, replay, coverage state, effects, cross-domain joins, and canonical backend persistence.

Beads + agents

Beads owns intended work and dependencies. Agents act, analyze, and propose. Neither agent prose nor task status substitutes for observed evidence.

Provider artifacts -> Sinex material/history -> Polylogue projections and UX -> reviewed context -> next agent

Sinex owns durable persistence. Polylogue owns AI-work meaning. SQLite remains the standalone authority or backed-mode edge replica.

A stranger can now see the thesis in one command.

The proof is deliberately small, falsifiable, provider-shaped, and explicit about what it does not establish.



```
polylogue demo receipts

1. THE CLAIM
"All tests pass."

$ polylogue demo receipts

Assistant · 14:32:20
All tests pass. The clock fix is complete.
Question: what evidence existed at that boundary?

The assistant's wording is a claim, not the oracle
```

At the claim boundary

Focused test receipt: exit 1, `is\_error=true`.

Later recovery

A later rerun exits 0; history is not rewritten.

Anti-grep control

Two prose hits for "error"; zero failed actions.

Clean source path

No-cache source build + proof: 14.75 seconds.

Verdict: contradicted_at_claim_time_then_repaired. This is a contract proof, not a prevalence or uplift claim.

Impressive demos come from stronger oracles, not louder animation.

Each rung proves a different construct; a launch claim may not climb higher than its evidence.

1	CONTRACT	Deterministic provider-shaped corpus	parser/storage/query/ref path
2	INDEPENDENT ORACLE	Frozen external source materials	product output cannot define truth
3	PUBLIC FIELD PACKET	Real merged change, privacy-safe	claim, receipt, Git/CI outcome
4	RUNNING JOINT STACK	Sinex admission and replay	coverage, settlement, rebuild
5	BLINDED EXPERIMENT	Raw vs summary vs reviewed context	predeclared scoring and stale traps

Rejected hero demos: architecture flyovers, dashboard tours, uncalibrated “AI insights,” and memory demos scored by eloquence.

A shared proof world independent of both products.

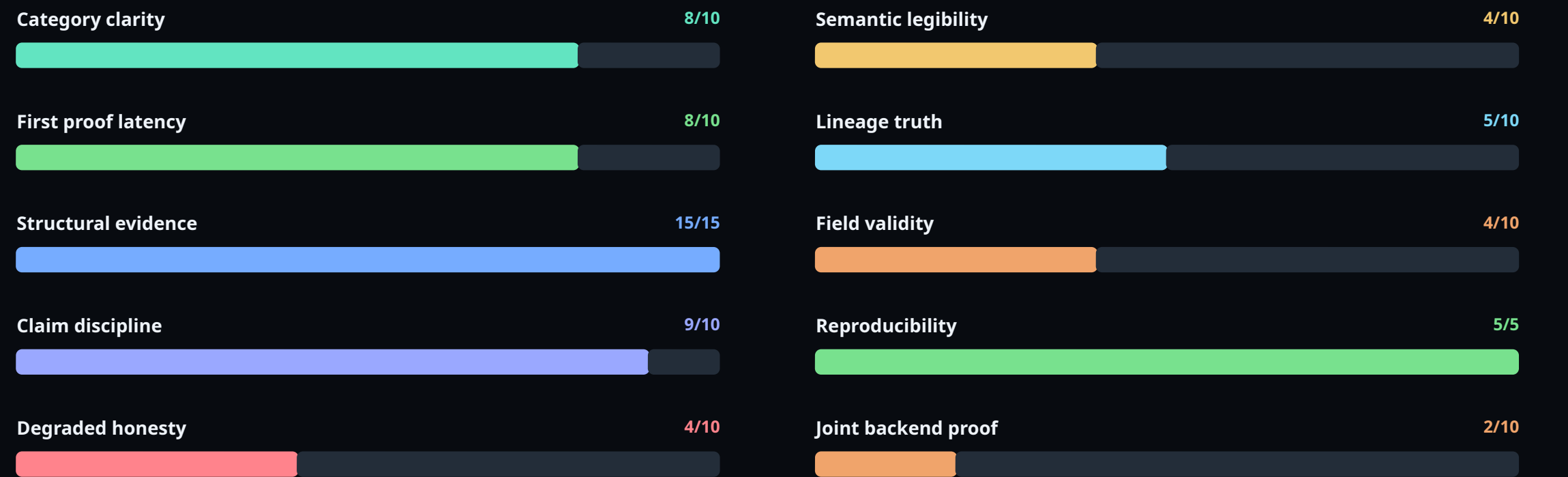
The corpus is small enough to inspect manually and rich enough to exercise evidence, lineage, coverage, judgment, context, and replay.

CLAIM	"All tests pass" at 14:32:20	RECEIPT	test exits 1 before the claim
REPAIR	later test exits 0	LINEAGE	11,200 physical vs 7,200 logical input tokens
COVERAGE	browser interval is unobserved, not empty	REPLAY	parser v2 supersedes v1; stable object ID survives
JUDGMENT	rejected candidate cannot enter context	CONTEXT	delivery records omissions explicitly

Verifier result: 15/15 material hashes, 24/24 oracle facts, zero public-safety findings.

64 / 100 - materially improved, still honestly pre-launch.

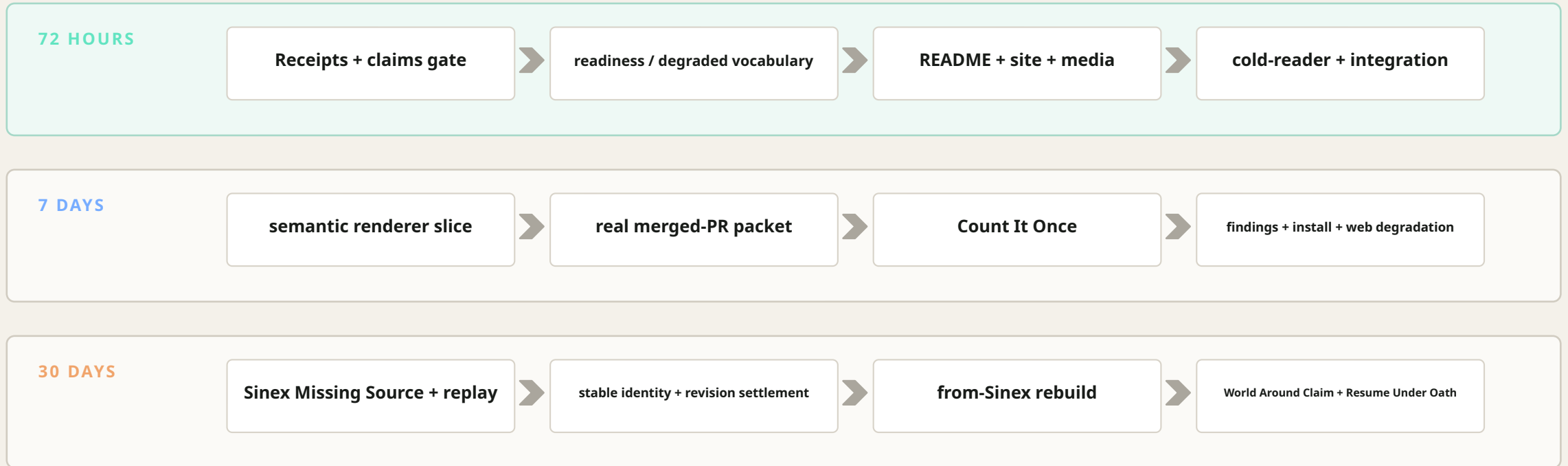
The score weights what an outsider can understand and verify, not internal code volume.



The remaining points are concentrated: truthful degraded states, semantic rendering, public field proof, and the Sinex-backed rebuild.

Twenty-three missions across three horizons.

Every mission has Beads owners, dependencies, write paths, resources, verification, a prompt, and a stop condition.



Integration missions are the only wildcard writers. Public copy cannot outrun the executable proof it describes.

Fast parallelism with explicit scarce-resource control.

Git history may be ugly; authority, evidence, and integration state may not be.

Python light	4	Python heavy	2
Rust heavy	1	Postgres/NATS	1
Browser/media	1	Full verification	1
Integration	1	Docs/review	6

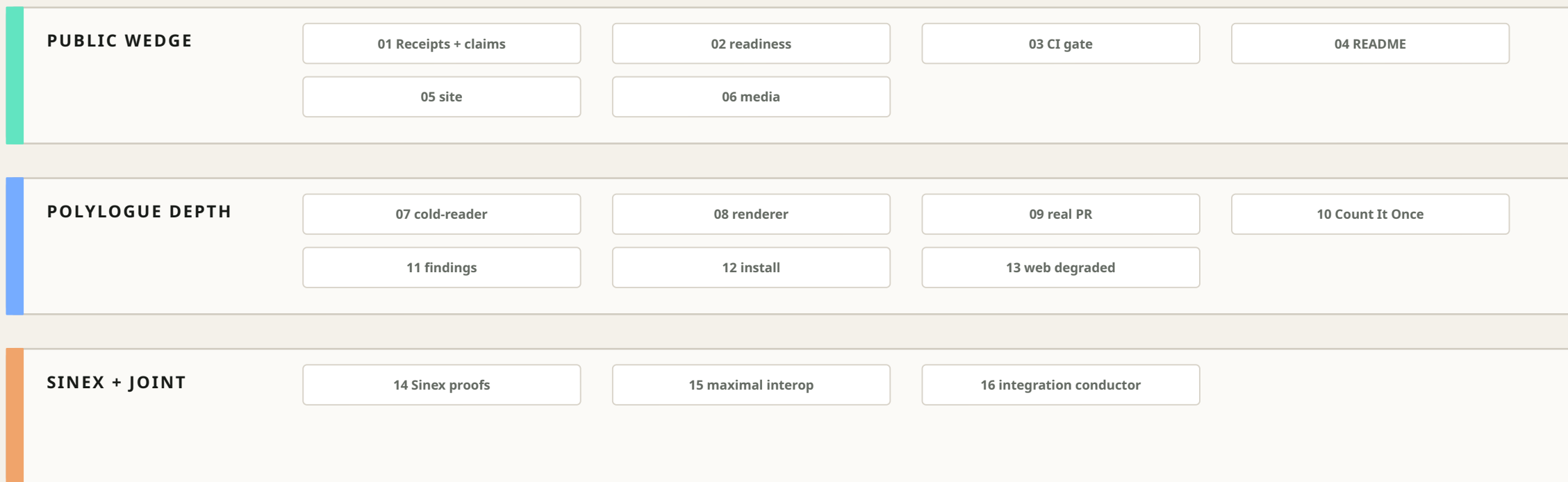
LEASED MISSION LIFECYCLE

- 1 claim mission + resources
- 2 create isolated worktree
- 3 heartbeat while active
- 4 write exact handoff
- 5 finish only after verification
- 6 coordinator integrates or rejects
- 7 reap expired leases

Overlapping write paths block claims. Stale leases are reaped. Handoffs without verification, failures, or merge guidance are rejected.

Each fork is a complete mission, not a vague request for help.

The prompts can be run as forks of this conversation and return patches plus evidence-rich handoffs.



Every prompt names owned paths, relevant Beads, controls, falsifiers, tests, artifacts, and the required handoff shape.

The integration is a backend contract, not a metadata notification.

The hard part is stable identity and complete revision settlement, not copying rows.

SINEX AUTHORITY

- provider-native bytes
- normalized immutable segments
- durable domain history
- judgments + lifecycle
- model effects + deletion

POLYLOGUE AUTHORITY

- origin/provider semantics
- sessions/messages/blocks/tools
- lineage + compaction
- physical/logical accounting
- memory/context/product UX

SQLITE EDGE ROLE

- standalone authority
- offline replica + outbox
- local FTS/vector cache
- deterministic demo backend
- local presentation state

Four identities stay separate: stable object ID, object revision, Sinex interpretation event ID, and exact material occurrence ref.

Presentable means credible on first contact.

The next public cut should ship only when the green checks are true. The ambitious backend and memory experiments remain visible, but do not block the first honest release.

✓	First proof from clean source	PASS
✓	Claims ledger in CI/release gate	PASS
✓	Independent oracle corpus	PASS
-	Semantic shell/edit/lineage cards	NEXT
-	Truthful empty/partial/stale/timeout UI	NEXT
-	Public-safe real merged-PR packet	NEXT
-	Transcript-complete Sinex rebuild	30D
-	General memory uplift	EXPERIMENT

Artifacts included: patches, proof packets, schemas, mission DAG, coordinator scripts, 16 prompts, deck, and validation report.